

MolSurf

Quick Reference

March 12, 2013

Quick building Guide

MolSurf builds using CMake which requires a version greater or equal to 2.8.x. Currently we distribute the source of MolSurf along with source distributions of all the third-party libraries that are required and tested. Although this can make our source package too heavy, we believe the most tricky part for users is to build and link all the dependencies on their specific platform, thus better to minimize that endeavor. By packing all the dependencies together and providing building environment for them, building MolSurf became relatively easy, we believe. Here is a very few steps to build our source package.

0.1 With ccmake

```
$ cd build-dir
```

```
$ ccmake source-dir
```

In ccmake GUI, set:

- CMAKE_BUILD_TYPE to *Release* or *Debug* (this should be consistent for third-party and MolSurf)
- PRE_BUILD to *ON*

Then configure (press ‘c’) and generate (press ‘g’) to generate makefiles for third-party libraries.

```
$ make
```

Once you have all the libraries built successfully, you need to configure MolSurf using ccmake.

```
$ ccmake source-dir
```

In ccmake GUI, set:

- CMAKE_BUILD_TYPE as same as in the third-party build
- PRE_BUILD to *OFF*
- CMAKE_INSTALL_PREFIX to your desired installation path

If you configure (press ‘c’) after setting PRE_BUILD to OFF, then cmake will try to set up MolSurf building environment including paths for third-party libraries you just built. If there is no error produced by cmake, then you should be good to go. But you may need to confirm the configuration by reconfiguring (press ‘c’) few more times. If the configuration is done, generate makefiles (press ‘g’) and execute ‘make’.

```
$ make
```

Then it should build MolSurf binary for you. The brief description of supported functions by MolSurf and their usage can be found below. The detailed description and examples can be found from MolSurf Reference Manual.

0.2 Without cmake

We recommend using ccmake to see all the available cmake options. If you have a problem running ccmake, you can use cmake without GUI. Basically you need to do the same things in the case of using ccmake by passing required arguments to cmake command line for proper configuration and makefile generation.

```
$ cd build-dir
```

```
$ cmake source-dir -DCMAKE_BUILD_TYPE=Release -DPRE_BUILD=ON
```

```
$ make (build third-party)
```

```
$ cmake source-dir -DCMAKE_BUILD_TYPE=Release -DPRE_BUILD=OFF
```

```
$ make (build MolSurf)
```

MolSurf Usage

```
MolSurf -h | --help
    Display this summary.
```

0.3 Surface Generation

```
MolSurf -surfaceUsingAdaptiveGrid <pdb/pqr input file> <raw output file>
      <size> [ <enlarged width> ]
    Generate solvent excluded molecular surface using the adaptive grid algorithm.
```

```
MolSurf -surfaceUsingGaussianBlur <pdb/pqr input file> <raw output file>
      <isovalue> <size> <blobbiness> <radius type> <level> <optimizedRadii>
    Generate molecular surface using the Gaussian blur algorithm.
```

```
MolSurf -surfaceUsingHLS <pdb/pqr input file> <raw output file>
      <size> [nIterations] [weight]
```

Generate molecular surface using the higher order level-set algorithm.

```
MolSurf -surfaceUsingLBIE <meshtype> <isovalue> <rawiv input file>  
      <raw output file>
```

Generate tri/quad/tet/hex mesh from volumetric data using the LBIE library.

```
MolSurf -convertToRawnUsingHLS <pqr/pdb input file> <rawn output file> <outfile2> <size> [n]
```

Generate molecular surface with surface normal vectors using the higher order level-set algorithm.

```
MolSurf -surfaceUsingFastSurf <pdb/pqr input file> <dim x> <dim y> <dim z> <bool SAS> <raw o  
      Generate molecular surface using a fast algorithm
```

```
MolSurf -surfaceUsingCardinalBSpline <pdb/pqr input file> <size> <int order(0-4)> [isovalue  
      Generate molecular surface using cardinal B spline algorithm
```

0.4 Area and Volume

```
MolSurf -area <raw input file> <txt output file>  
      Compute the surface area of a triangular mesh.
```

```
MolSurf -volume <raw input file> <txt output file>  
      Compute the volume bounded by a triangular surface mesh.
```

0.5 Normal

```
MolSurf -normals -average <raw input file> <rawn output file>  
MolSurf -normals -loop <raw input file> <rawn output file>  
      Compute surface normals at vertices of a triangular mesh using face normal  
      averaging or Loop's algorithm
```

```
MolSurf -normals -invert <rawn input file> <rawn output file>  
      Invert surface normals
```

0.6 Analytic Spline

```
MolSurf -aspline -raw <raw input file> <raw output file> <resolution>  
      Generate A-spline model (ASMS) from a triangular mesh.
```

```
MolSurf -aspline -quadrature <raw input file> <quadrature rule>
```

<points per triangle> <txt output file>
Generate quadrature points for an ASMS from a triangular mesh.

MolSurf -aspline -volume <raw input file> <txt output file>
Compute the volume bounded by an ASMS from a triangular mesh.

0.7 Coarse Grain

MolSurf -coarseGrain <radius type: pqr or vdw> <input pdb/pqr> <pqr output> <cg info output>
Generate coarse grain pqr

MolSurf -optimizedCGCharge <size> <iteration> <ARM> <heirarchical information> Optimiz
information.

0.8 Volume Generation

MolSurf -volumeUsingHLS <pdb/pqr input file> <rawiv output file>
 <size> [nIterations] [weight]
Generate volumetric level-set function.

MolSurf -signedDistaceFunction <raw input file> <rawiv output file> <size>
Signed distance function using multi-sdf

MolSurf -signedDistaceFunction2 <raw input file> <rawiv output file> <size x> <size y> <size z>
Signed distance function using multi-sdf with more options

MolSurf -getVolumeWithHydrophobicity <pdb/pqr input file> <rawiv output file> <dim x> <dim y> <dim z>
Compute volume with hydrophobicity

MolSurf -getVolumeWithElectron <pdb/pqr input file> <rawiv output file> <dim x> <dim y> <dim z>
Compute volume with electron

MolSurf -getVolumeWithChargeDensity <pqr input file> <rawiv output file> <dim1> <dim2> <dim3>
Compute volume with charge density

0.9 Operation on Mesh

MolSurf -qualityImprove <raw input file> <raw output file>
Improve the mesh quality using geometric flow.

MolSurf -decimate <raw input file> <raw output file>
 Improve the mesh quality using geometric flow.

MolSurf -decimatePercentage <raw input file> <raw output file>
 Improve the mesh quality using geometric flow.

MolSurf -reoriente <input geom. file> <output geom. file>
 Change the orientation of the surface

MolSurf -offsetSurface <rawn input file> <rawn output file> <offset distance>
 Create an offset mesh

MolSurf -colorByCharge <pqr file> <rawn input file> <rawnc out file> <0=gaussian, 1=average>
 Color mesh by atomic charge

MolSurf -getInterfaceSurface <receptor rawn/rawnc file> <ligand rawn/rawnc file> <interface>
 Get interface surface

MolSurf -removeInteriorPockets <raw input file> <raw output file>
 Remove small pieces interoir the surface

MolSurf -partitionMesh <raw input file> <# of partitions>
 Partition mesh for parallel computation

MolSurf -repairSurface <raw input file> <raw output file> [epsilon]
 Remove duplicate vertices from input mesh

MolSurf -mergeGeometry <input 1> <input 2> ... <output file>
 Mergy input geometry files into single file

MolSurf -colorMeshByAtomType <pdb input file> <raw input file> <rawn out file>
 Color mesh by input atom type

0.10 Pocket and Tunnel

MolSurf -pocket <pdb file> <distance> <size> <out file>
 Compute pockets for given pdb

MolSurf -pocketTunnel <input geom. file> <opc> <otc> <out file1> <out file2>
 Compute pockets and tunnels for given surface geometry

0.11 Opertation on PDB/PQR

MolSurf -populateSAS <pdb/pqr file> <out file> <size> <prob radius> <gap between surface and atoms>
Populate atoms to SAS

MolSurf -populateSASUsingMesh <pqr file> <rawn file> <xyz out file> <probe radius> <gap between surface and atoms>
Populate atoms to SAS using mesh

MolSurf -getInterfaceAtoms <receptor pqr file> <ligand pqr file> <txt out file> <interface width>
Get interface atoms

MolSurf -leastRMSD <pdb input 1> <pdb input 2>
Compute RMSD

MolSurf -rotatePDB <pdb input file> <transformation file> <pdb out file>
Rotate a molecule

MolSurf -surfaceAtoms <pdb input file> <output file>
Compute which atoms of a molecule touch the molecular surface

MolSurf -mergeProtein <radius type: vdw / pqr> <input file 1> <input file 2> <output file>
Mergy two protein files

MolSurf -splitPDB2Chains <pdb input file> <output file (without ext.)>
Split input pdb to changes

MolSurf -PQR2PDB <with(1)/without(0) hydrogen> <with(1)/without(0) chain> <pqr input file> <output file>
Convert pqr to pdb

MolSurf -PQR2XYZR <pqr input file> <xyzr output file>
Improve the mesh quality using geometric flow.

MolSurf -tinkerPDBXYZ2PQR <pdb input file> <xyz input file> <Born radius input> <pqr output file>
InputPDB is the output of Tinker xyzpdb

0.12 Un-categorized

MolSurf -bornRadius <radius type: vdw or pqr> <input pqr> <surface geometry file> <output file>
Compute atomic Born radius for a given surface

MolSurf -hausdorff <raw/rawn input file> <raw/rawn input file 2> <1=raw / 2=rawn>
Compute hausdorff for given meshes.

```
MolSurf -generateF2d <pqr input file> <xyz file> <out file> <int receptor=1/ligand=0> <radius>
Generate F2d???
```

```
MolSurf -atom2surfaceDistance <pqr input file> <raw input file> <cdr out file>
Compute atom-surface distance, output file formed pqr, with color at position, distance
```

1 MolSurf Libraries

MolSurf is made up of a variety of code libraries that can be interfaces directly.

- AdaptiveIsocontouring
- ASpline - Construct and manipulate A-spline surface models.
- Blurmaps - Construct Gaussian molecular surfaces.
- ByteOrder
- C2C
- CardinalBSpline
- ComputeNormals - Routines for computing normals to surfaces meshes.
- Contour
- Contouring
- Curvatures
- Decimation
- Geometry
- GeometryFileTypes
- GOAFileTypes
- HLevelSet
- LBIE_lib - Level-set boundary interior exterior mesh library.
- levmar-2.2
- libCG - Routines for constructing and optimizing coarse-grained molecular models.
- MolecularCharacteristics
- MolSurf - Main MolSurf commandline interface.
- MultiresolutionSurface

- PDBParser - Read PDB files to usable data structures.
- Pocket -
- PocketTunnel
- SignDistanceFunction
- SignDistanceFunction_v2
- SmoothingFunction
- SphericalPatchIntersections
- Summation
- SurfaceDataManager
- UsefulMath - Some generally useful routines.
- VolumeFileTypes